

Programming Assignment 1

Finding strongly connected groups in a directed graph

Kosaraju's algorithm and an empirical comparison with Tarjan's algorithm

Jonathan Immanuel

Student ID: 14487692

Track A: Implementation and empirical study

Code and results: github.com/jo-nuel/kosaraju-scc

Table of Contents

1. What I built	3
How the algorithm is organised.....	3
The comparison and checks.....	3
2. Empirical study.....	4
Question and expectations.....	4
How I made the graphs.....	4
How I measured.....	4
3. Results and interpretation.....	5
Running time.....	5
Process memory	7
What these results do not show.....	7
4. What I learned	8
5. AI use	9
Tools and what I used them for	9
Where the AI was wrong or unhelpful.....	9
What I understand and what I still rely on	9
Sources and reproducibility	9

1. What I built

A directed graph is a collection of places joined by one-way links. A strongly connected component is a group where every place can reach every other place in that group. My program finds these groups with Kosaraju's algorithm. I used C++23 because the graph can be stored in ordinary vectors and the same code can handle both small examples and large tests.

The graph is kept as an adjacency list: each vertex has a list of places its outgoing edges lead to. The main result gives each vertex a component number and gives the total number of components. Component numbers are just labels. Two correct programs may give the same groups different numbers.

How the algorithm is organised

The first search records when each vertex finishes. The code then makes a second graph with every edge reversed. A second search visits that reversed graph in the opposite of the finishing order. Every new search in this second pass marks one complete component. The implementation is in `src/kosaraju.cpp`; the graph and edge reversal are in `src/graph.cpp`.

The searches use explicit stacks rather than recursive calls. A very long path can exhaust the program's call stack. In the first pass, each stack entry also remembers which neighbour to inspect next. That detail lets the program record a vertex only after its descendants have finished, which is the order Kosaraju needs.

The comparison and checks

For a baseline, I used Tarjan's algorithm in `src/tarjan.cpp`. It uses the same graph type and result format, but finds components in one search by keeping visit order, a value showing how far back a path can reach, and a stack of unfinished vertices. It also uses explicit search frames. The benchmark code is in `benchmarks/`, the sweep and plotting scripts are in `scripts/`, and the raw results are in `results/`.

The tests cover the small hand-worked example, unusual edges, long paths, graph generators, and all 65,536 possible directed graphs on four vertices. For those small graphs, the test checks the grouping against mutual reachability rather than comparing component numbers. The C++ test program passed before the recorded benchmark run.

2. Empirical study

Question and expectations

I compared Kosaraju with Tarjan on the same directed graphs. Both have running time proportional to the number of vertices plus the number of edges, but they do different work. Kosaraju makes a reversed copy of the graph and searches twice. Tarjan searches once while tracking more information. I expected both to get slower as graphs grew. I expected Tarjan to use less memory, but thought Kosaraju might still be competitive in time on some graphs.

How I made the graphs

I used five graph families so the result would not depend on one shape. Paths have one-way progress and many separate components. Cycles put all vertices in one component. Sparse random graphs have four edges per vertex. Dense random graphs use about 25% of all possible directed edges. Clustered graphs use groups of ten vertices, with a cycle and 30 extra internal edges in each group, then one forward-only bridge to the next group.

The random generator excludes self-loops and repeated edges so an edge count means that many different connections. It uses seed 14487692, so the same settings make the same graph again. During timing, the graph is built once and handed unchanged to both algorithms for all repeated runs.

How I measured

The timed section contains only the algorithm call. Each algorithm gets one warm-up run and seven recorded runs. The order of the algorithms alternates between repetitions. I saved every run in a CSV file and used the median for the plots. This reduces the effect of one unusually slow run. The number of components was checked across runs.

Memory was measured differently. A fresh Windows process rebuilt the same graph for each algorithm and reported its peak working set after the algorithm finished. That is the highest process memory reading, including the stored graph and program. It is not a count of memory allocated by the algorithm alone. There is only one such reading per algorithm and graph case.

The benchmark used a Windows release build with MSYS2 UCRT64 GCC 16.1.0, C++23, and -O2. The bounded sweep ran 26 graph cases: paths and cycles up to one million vertices, sparse random graphs up to 256,000 vertices, dense random graphs up to 3,200 vertices, and clustered graphs up to 320 groups. The two ten-million-vertex cases and the 6,400-vertex dense case were not run because their memory needs had not been checked on this computer. Those cases are not part of the results.

3. Results and interpretation

The table shows the largest measured case in each graph family. Times are medians of seven runs. Memory is one peak process reading. MiB means 1,048,576 bytes.

Graph	Vertices	Kosarajums	Tarjanms	KosarajuMiB	TarjanMiB
Path	1,000,000	81.50	21.45	97.3	90.3
Cycle	1,000,000	59.42	22.03	97.3	90.2
Sparse random	256,000	182.62	65.59	82.0	70.8
Dense random	3,200	35.07	5.40	133.2	133.2
Clustered	3,200	0.65	0.13	5.9	5.8

Table 1. Largest measured case from each family. The families use different vertex and edge ranges, so a time in one row should not be treated as a direct same-size comparison with another row.

Tarjan had a lower median running time in all 26 measured cases. On the one-million-vertex path, Tarjan took 21.45 ms and Kosaraju took 81.50 ms. On the largest dense random graph, the medians were 5.40 ms and 35.07 ms. The plots below show selected graph families. The complete five-family figures and every recorded time are in the repository.

Running time

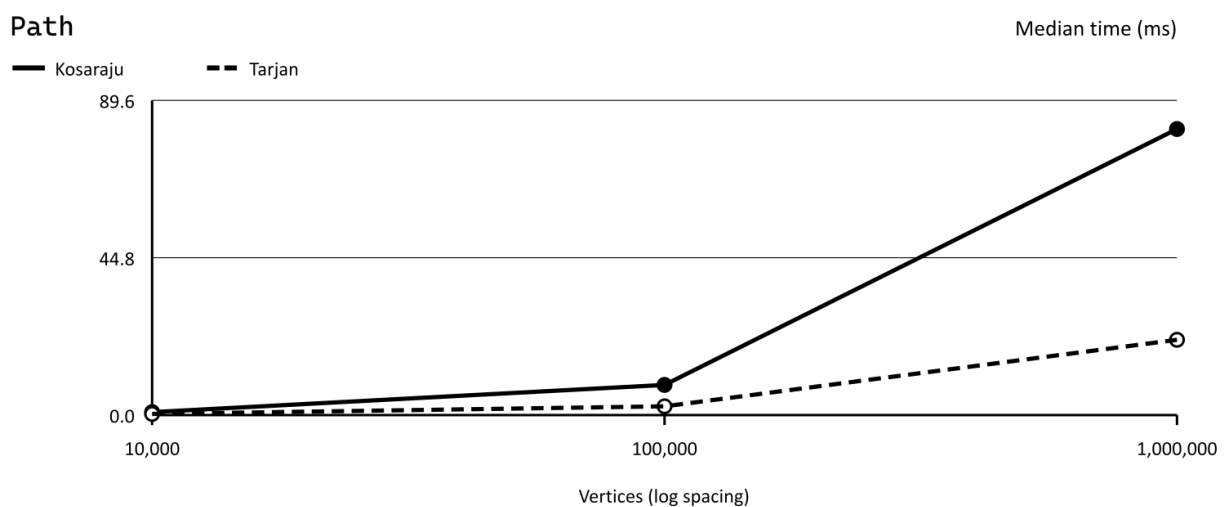


Figure 1. Median running time on paths. Larger paths took longer for both algorithms.

Sparse random

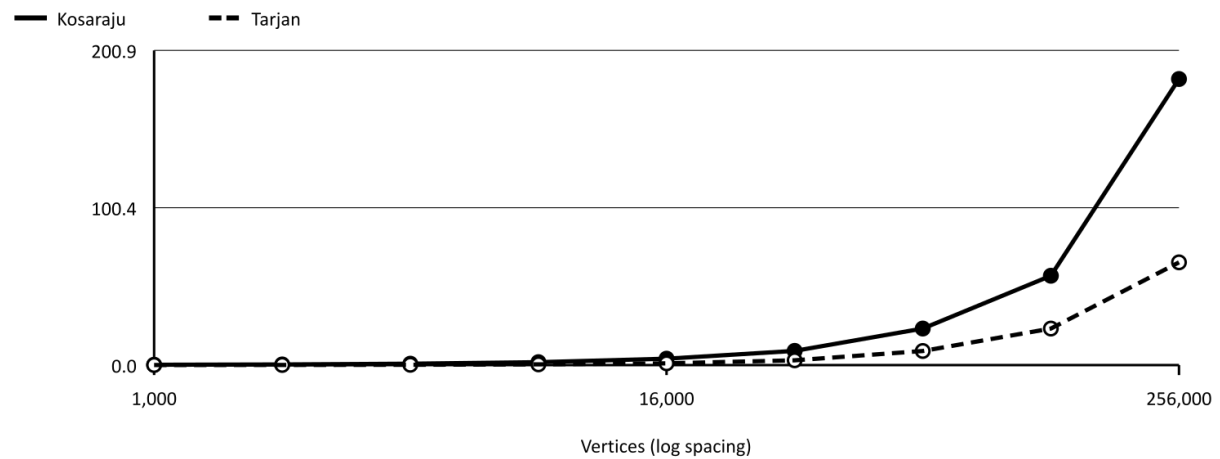


Figure 2. Median running time with four directed edges per vertex.

Dense random

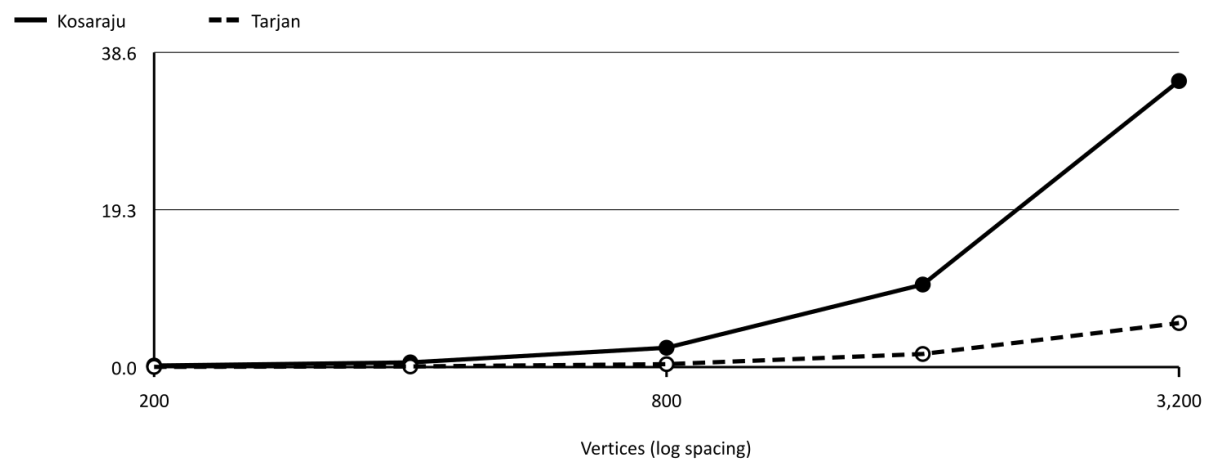


Figure 3. Median running time at about 25% edge density. The vertex range is smaller than for paths or sparse graphs.

The increase in time as these graphs grow is consistent with both algorithms doing more work when there are more vertices or edges. It does not by itself prove a precise growth rate. A likely reason for Kosaraju's larger times here is the extra reversed graph and second search. The benchmark did not time those steps separately, so it cannot show exactly how much each step contributed.

One dense Tarjan repetition took 9.51 ms, while its median was 5.40 ms. Keeping all seven raw times made that difference visible. The median prevented that one run from setting the headline result.

Process memory

Sparse random

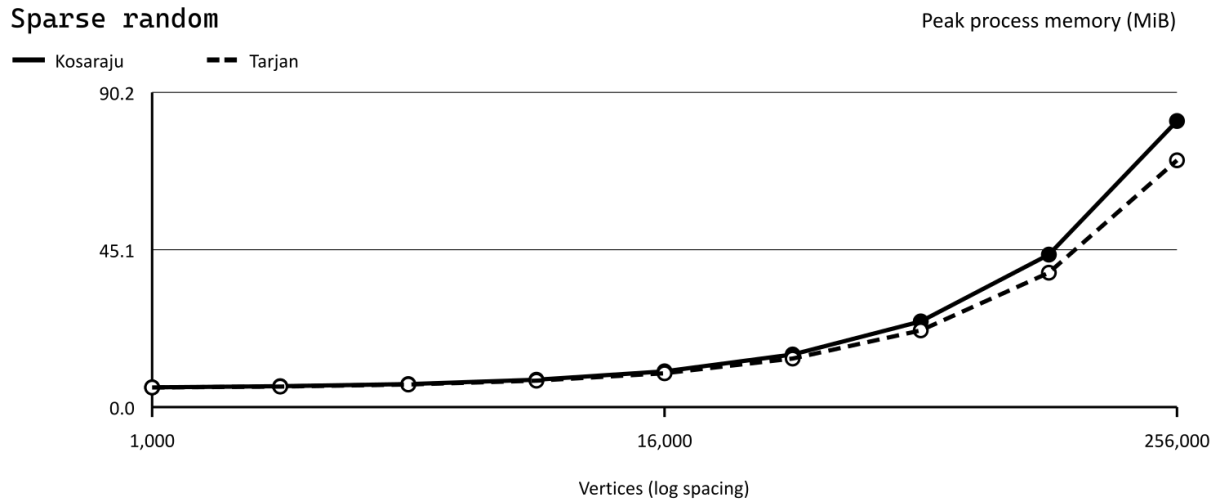


Figure 4. Total peak process working set on sparse random graphs.

Dense random

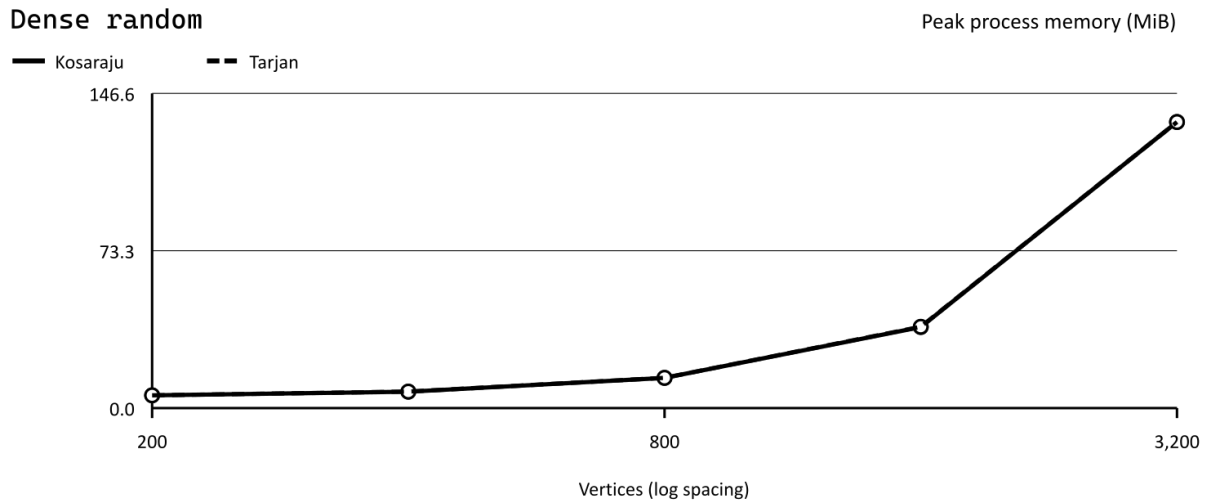


Figure 5. Total peak process working set on dense random graphs.

On the largest sparse graph, Tarjan's process peaked at about 70.8 MiB and Kosaraju's at 82.0 MiB. On the largest dense graph, both were about 133.2 MiB. This suggests that storing the dense graph can hide smaller differences between the algorithms. Since each peak includes more than the algorithm and was measured once, I would not treat tiny gaps as meaningful.

What these results do not show

These 26 cases were measured on one computer with one compiler setup. The sweep did not include the three largest candidate cases. Hardware details such as the processor model were not recorded. The results support saying Tarjan was faster on the tested graphs, not that it is always faster on every graph or system. They also cannot tell me each algorithm's exact extra memory use.

4. What I learned

I expected the two algorithms to have similar-looking times on at least some graph shapes, because both have the same overall time bound. That did not happen in this run. Tarjan's median was lower in every case, sometimes by several times. I now understand that a shared growth bound does not mean two implementations do the same amount of work. Kosaraju's reversed graph and second search matter in practice, even though they do not change the overall bound.

The first version of Kosaraju used recursive search. A long path made the call-stack limit a real problem, so the search was changed to keep its own stack. The tricky part was preserving the true finishing order. Remembering the next neighbour in each stack entry solved that. I understand now why simply pushing vertices when I first see them would be wrong for Kosaraju's first pass.

I also mixed up Tarjan's visit number and its low-link value when I first traced the small example by hand. The visit number stays fixed. The low-link value can change if a path reaches a vertex still being worked on. In the example, vertex 4 points back to vertex 3, so those two end up in the same component. I checked the final grouping against the tests, although I did not check every intermediate value against printed program output.

The memory work changed how I describe a measurement. I first thought of it as extra algorithm memory, but the Windows reading covers the whole process. Reviewing the dense results made that difference clear: both processes reached nearly the same peak even though their times were far apart. The number is useful as a rough comparison, but it answers a narrower question than exact algorithm allocation.

5. AI use

Tools and what I used them for

I used ChatGPT heavily across the project. It helped plan the repository, scaffold and revise the C++ implementations, write tests, set up the benchmark and graph generators, automate the CSV sweep, make the plots, and troubleshoot build and Git problems. I chose Kosaraju and Track A, worked out graph and benchmark rules, traced the example, and reviewed the CSV values and plots. The recorded benchmark sweep was run through ChatGPT, I inspected its outputs rather than running another full sweep myself.

Where the AI was wrong or unhelpful

First, the early Kosaraju code used recursive depth-first search. That looked fine on small graphs but was unsafe on a long path because the call stack could run out. A long-path test exposed the problem. The code was changed to explicit stacks, and the tests were run again. This was not just a style change: the new stack had to keep the correct finishing order.

Second, ChatGPT initially said that a native C++ compiler was unavailable after checking only the normal command path. A later search found GCC in the MSYS2 installation. The first build still failed because the compiler could not find a runtime DLL. Adding the MSYS2 runtime folder to that build process's path let the native tests run. The lesson for me was to check the environment more carefully before accepting a claim that a test cannot be run.

There was also a false alarm in an early JavaScript correctness checker. It compared Boolean values with 0 and 1 using strict equality and reported a mismatch that was caused by the checker, not by the SCC code. The check was corrected before its result was trusted. I treated that JavaScript check as extra evidence only, the later native C++ tests were the main correctness check.

What I understand and what I still rely on

I can explain the two passes of Kosaraju, why the second pass uses reverse finishing order, how the graph families were built, why both algorithms need the same input, and why the median is used. My hand trace helped me understand the Tarjan example and I checked the final groups. I still rely on the automated tests for larger and unusual graphs that I have not traced myself. I also rely on the benchmark code and Windows process counter for the recorded numbers. I checked the CSVs and plots, but I did not independently measure the algorithms with another profiler. I would not claim the memory figures show exact algorithm allocations.

Sources and reproducibility

The code, build instructions, benchmark scripts, raw CSV files, and full five-family figures are in the public repository: github.com/jo-nuel/kosaraju-scc (main branch, commit 9a5c20a). The measured run is dated 15 September 2026 in results/run-notes.md. This report follows the supplied Programming Assignment 1 description and my dated review notes.